

Training a Code LLM from Scratch • Part 3 (Detailed)

Distributed Training & Long Context: A Full Derivation with All the Back-and-Forth

From "why one GPU can't hold it" to TP / EP / FlashAttention / Ring / MLA, using 31B dense / 671B MoE / 1M-100M context as yardsticks. **Orange blocks record the back-and-forth in the derivation (wrong assumption → correction → correct understanding) — the most valuable part.**

Outline (10 steps, each with derivation back-and-forth)

- **1.** Memory breakdown: why one GPU can't hold 31B
- **2.** Tensor Parallel TP (back-and-forth: GQA dims, inner-product downscaling, up/down-projection communication)
- **3.** Gradient checkpointing (back-and-forth: what's the big activation, what to store, recompute granularity)
- **4.** Data Parallel DP
- **5.** Expert Parallel EP (most back-and-forth: TP/EP confusion, move token vs expert, all-to-all vs ring)
- **6.** FlashAttention (full pseudocode + multi-head + back-and-forth: KV-cache analogy, exact or approximate, online softmax bug)
- **7.** Ring Attention (full pseudocode + seq-split $\times$ head-TP + back-and-forth: ring send vs all-reduce)
- **8.** 100M: change the algorithm
- **9.** KV cache economics at scale
- **10.** MLA (back-and-forth: same as GQA? PCA? attention-matrix factorization?)

1. Memory breakdown: why one GPU can't hold 31B

Yardstick: Gemma-4-31B — 60 layers, $d_{\text{model}}=5376$, FFN inner dim 21504 (GeGLU), 32 heads ($\text{head_dim}=256$), GQA KV heads 16, vocab 262144, tied embedding.

Per-layer parameters

Part	Matrix	Params
Attention	$W_q\ 5376 \times 8192$	44M
	$W_k\ 5376 \times 4096$	22M
	$W_v\ 5376 \times 4096$	22M
	$W_o\ 8192 \times 5376$	44M

FFN	gate 5376×21504	116M
	up 5376×21504	116M
	down 21504×5376	116M

Per layer $\approx 479\text{M}$ (FFN is 72%). 60 layers $\approx 28.7\text{B}$ + embedding ($\approx 1.4\text{B}$, tied) $\approx 30.7\text{B}$.

1.3 Full inventory: params, grads, optimizer, activations

30.7B full-param training, BF16 mixed precision, item by item:

- **Params (BF16)**: forward/backward, $30.7\text{B} \times 2 = 61.4\text{G}$.
- **Gradients (BF16)**: one partial $\partial L / \partial w$ per param; gradient tensor has the **same shape** as params, $30.7\text{B} \times 2 = 61.4\text{G}$.
- **AdamW first moment m (FP32)**: EMA of gradients (direction momentum), $30.7\text{B} \times 4 = 122.8\text{G}$.
- **AdamW second moment v (FP32)**: EMA of squared gradients (adaptive scaling), $30.7\text{B} \times 4 = 122.8\text{G}$.
- (possible FP32 master params: 122.8G, some implementations)
- **Activations (BF16)**: see § 3, $\sim 10\text{-}40\text{G}$ with grad_ckpt, 78G+ without.

Fixed overhead (excl. activations): $61.4 + 61.4 + 245.6 = 368.4\text{G}$ (+master $\rightarrow 491\text{G}$). With activations $\rightarrow \sim 400\text{G}+$.

Why m, v are FP32: they are **accumulated statistics** (EMA); BF16 accumulation loses small updates (large-number-eats-small). FP32 preserves accumulation precision. This is the memory hog of full-param training ($m+v = 245.6\text{G}$, bigger than params+grads).

Corollary: QLoRA saves because the base is frozen (no grad/optimizer needed); only the small LoRA ($\sim 0.7\%$ of params) has grads and AdamW state, shrinking the 245.6G hog to a few G.

1.4 Inference vs training memory

	Inference	Training (full)
Params	61.4G	61.4G
Gradients	—	61.4G
AdamW (m,v)	—	245.6G
Activations	small	78G+ (or grad_ckpt 10-40G)
KV cache	yes ($\propto$ context)	none (full seq at once)
Total	$\sim 80\text{G}$ (single card)	$\sim 400\text{G}+$ (multi-card/QLoRA)

Back-and-forth ① (AdamW vs Adam) — Confirmed it's **AdamW** (modern LLM standard). AdamW differs from Adam only in decoupled weight decay (update formula); **optimizer state is the same size** (m,v, 245.6G), no memory difference.

2. Tensor Parallel TP: split by head

```
Input X: (b, s, 5376)
Q=X·Wq(5376, 8192)  K=X·Wk(5376, 4096)  V=X·Wv(5376, 4096)
A=softmax(QKT/√d)  H=A·V  O=H·Wo(8192, 5376)  X1=X+O
FFN: G=X1·Wg  U=X1·Wu(5376, 21504)  M=gelu(G)⊙U  D=M·Wd(21504, 5376)  X2=X1+D
```

Back-and-forth ② (GQA dims) — First guessed Wk/Wv's 2nd dim is 8192. **Correction:** under GQA, Q uses 32 heads (→8192), K/V only 16 heads (→**4096**). Fewer KV heads is how GQA saves KV cache.

Back-and-forth ③ (does the inner product need downscaling?) — Once thought "Wq must downscale first, else the inner product doesn't work". **Correction:** the inner product happens **inside each head**; Q's per-head 256 dim · K's per-head 256 dim, same head_dim, direct. Wq→8192 is because of 32 heads, Wk→4096 for 16 heads, but every head is 256, dims match — **no downscaling needed**.

Back-and-forth ④ (why 2 Q share 1 KV) — Not "need 2 Q", but 32 Q heads share 16 KV heads (32/16=2). **Motivation:** KV cache $\propto$ KV head count; fewer KV heads (16) halves cache (esp. long context); many Q heads (32) keep expressiveness. GQA is the middle ground between MHA (all 32, big cache) and MQA (1 KV head, expressiveness loss).

2.3 Split & communication: up-projection local, down-projection all-reduce

Each card holds the full X (same copy), weights split by head:

- **Up-projection** (Wq 5376→8192, Wg/Wu 5376→21504): each card locally computes $X \times$ its weight slice → its own heads' Q (or its FFN-inner slice), **no communication** (weights pre-split; up-projection output is naturally each-card-owns-a-slice).
- **Down-projection** (Wo 8192→5376, Wd 21504→5376): each card computes a **partial O_i** from its slice; full $O = \sum \text{partial } O_i \rightarrow$ **ring all-reduce (sum)**.

Back-and-forth ⑤ (up-projection local or centralized, repeatedly) — Went back and forth on whether 5376→8192/21504 is "each card local" or "one card computes the whole"; the wording once called "local (no comm)" as "one card". **Final clarity:** 5376→8192 and 5376→21504 are handled **identically** — both up-projections, each card computes its own slice (2688 or 1024 dim) locally, no comm. The only difference is slice size, not mechanism. **Up-projection (5376→big) is local no-comm; down-projection (big→5376) is all-reduce sum.**

Back-and-forth ⑥ (Wo all-reduce detail) — Each card's $H_i(b, s, 1024) \times W_{o_i}(1024, 5376) = \text{partial } O_i(b, s, 5376)$; 8 partials summed = full O. This uses **ring all-reduce**: 8 cards in a ring accumulate stepwise, ~8 steps and every card has the full sum (not gathered to one card then broadcast).

Back-and-forth ⑦ (recompute up-projection vs transfer) — Next layer's 5376→8192 each card recomputes its own slice locally (duplicate compute), rather than "one card computes then copies to others". The former saves the copy transfer at the cost of duplicate compute; for small projections, duplicate compute is cheaper than cross-card copy.

2.4 Step-by-step data flow (one layer, head TP, 8 cards)

From one card's view (4 heads, head_dim=256):

1. **Input:** X (b,s,5376), full copy on every card.
2. **Q projection (up):** card holds W_q slice (5376, $4 \times 256 = 1024$). $Q_local = X \cdot W_q_slice \rightarrow (b,s,1024)$ = this card's 4 heads' Q. **No comm.**
3. **K/V projection:** under GQA, 2 KV heads $\rightarrow W_k/W_v$ slice (5376,512), $K_local/V_local = (b,s,512)$.
4. **attention:** 4 Q heads paired with 2 KV heads (2:1, KV broadcast), per-head $QK^T \rightarrow \text{softmax} \rightarrow \times V \rightarrow H_local$ (b,s,1024). Each head independent.
5. **Output projection (down):** W_o row slice (1024,5376). partial $O_i = H_local \cdot W_o_slice \rightarrow (b,s,5376)$. **This is a partial sum** (only this card's 4 heads).
6. **all-reduce:** 8 cards' partial O_i summed = full O. Every card gets full O.
7. **Residual:** $X_1 = X + O$ (local; X and O both full).
8. **FFN:** same — W_g/W_u column-split (up local, G_local/U_local (b,s,2688)), W_d row-split (down $\rightarrow$ partial $D_i \rightarrow$ all-reduce). $X_2 = X_1 + D$.

Two all-reduces per layer (after W_o , after W_d). 60 layers $\rightarrow$ 120 all-reduces. This is why TP communicates frequently, and why TP must be intra-node NVLink (each all-reduce moves (b,s,5376)-scale data; only high intra-node bandwidth can handle it; cross-node network bottlenecks).

Params vs activations vs sequence length: params ($W_q/W_k/W_v/FFN$) are determined only by d_model , **independent of seq length** (all tokens share one set — reuse). Longer seq only grows activations ($s \times s$ attention matrix, per-token intermediates), not params. Training 60 deep layers works via **residual connections** ($x+f(x)$, gradient flows straight through).

3. Gradient checkpointing: the big activation is the s^2 attention matrix

Back-and-forth ⑧ (what's the big activation) — First judged "per-layer activation = layer output (b,s,5376) $\approx$ 29MB, 60 layers only 1.7GB, small". **Correction:** missed the **attention score matrix $A = QK^T$, shape (b,heads,s,s), $\propto s^2$** . $s=2048$, 128 heads: $128 \times 2048 \times 2048 \times 2 \approx$ **1GB/layer**. 60 layers without checkpointing $\approx$ **78GB** $\rightarrow$ adding params overflows 80G. **The big activation is the s^2 A, not the layer output.**

Standard: split 60 layers into segments, store only **segment-boundary inputs** (few checkpoints); intra-segment intermediates (Q/K/A/O/FFN) discarded after forward; on backward, **re-forward from the boundary** to reconstruct. Saves memory, costs ~25% (an extra forward).

Back-and-forth ⑨ (what does grad_ckpt store) — Once thought "store Q/K/O for BP". **Correction:** that's the **non-checkpointed** way (store all). grad_ckpt **doesn't store** Q/K/A/O; stores only segment-boundary inputs, recomputes on backward.

Back-and-forth ⑩ (what do the 10 points store) — Once thought "10 checkpoints store Q/K/A/O", computed 290MB. **Correction:** the 10 points store **segment inputs X** (~29MB each, 290MB total), **not Q/K/A/O**; Q/K/A/O stored nowhere, recomputed on BP. 290MB is resident; BP has transient peaks (recomputed segment's Q/K/A/O incl. the s^2 A, freed after).

Back-and-forth ⑪ (spotting the inefficiency → fine-grained improvement) — You noted: standard "every N layers per segment" has an inefficiency — on backward, later layers in a segment re-forward from the segment head (BP layer 19 from layer-10 head to 19, BP layer 18 again from 10 to 18), $O(\text{segment}^2)$ recompute. **Your improvement:** store every layer's input X ($60 \times 29\text{MB} \approx 1.7\text{GB}$, small) → on BP at a layer, recompute just that one layer's Q/K/A/O from its X, **each layer recomputes only itself**, minimal recompute. Essentially finer-grained checkpointing (one per layer).

Variant: store Q/K/O + layer IO, **only skip A (the s^2 hog)**, **recompute A** — essentially close to FlashAttention (don't store full A).

3.4 Precise activation memory estimate

Without grad_ckpt, per-layer activation ($b=8, s=4096, d=5376, \text{ffn}=21504$):

- Q/K/V/O intermediates: $b \times s \times (\text{proj dims}) \times 2$ bytes
- **attention scores A:** $b \times \text{heads} \times s \times s \times 2 = 8 \times 128 \times 2048 \times 2048 \times 2 \approx 1.07\text{GB}$ (the hog, $\propto s^2$)
- FFN intermediates (21504 dim): a few hundred MB

Per layer $\approx 1.3\text{-}1.5\text{GB} \rightarrow 60 \text{ layers} \approx 78\text{-}90\text{GB} \rightarrow$ single 80G is full on activations alone.

Theory vs measured differs 1.7×: PyTorch caching-allocator reservation + fragmentation + transient-tensor peaks + activation underestimate. DL memory theory only estimates the order of magnitude; exact values need measurement.

Trade-off: larger segments → store less but recompute more + higher peak; smaller (per-layer) → store more (1.7G) but minimal recompute. When X is small, fine-grained wins. BF16 is the memory-saving key (activation 2 bytes; FP32 doubles it 78→156G).

micro-batch × segment count inverse: activation $\propto$ batch × per-segment activation. micro-

batch=4 + 20 segments $\equiv$ micro-batch=1 + 5 segments, same memory (batch $\times$ 4 shrinks segment to 1/4).

4. Data Parallel DP

Each card (group) holds a model replica, processes different data (batch split), computes grads, all-reduce to sync. **Grads and AdamW state follow the params** — on the same card as the param, to update locally ($W \leftarrow W - lr \cdot f(m,v)$) without cross-card moves. Mismatched placement causes extra cross-card transfer.

In this MoE setup, **each card holding independent data is already DP**; each card's full X is this DP unit's data, no need to further split X.

Back-and-forth ⑫ (positioning DP) — Briefly stuck on "should X be DP-split again". Cleared up: MoE's per-card independent data is already DP; no "re-splitting X". Whether X is split depends on X size (§ 7: only when 1M makes X too big do we seq-split).

5. Expert Parallel EP: full X per card + local compute + move results

671B MoE: 61 layers, 256 routed experts (+1 shared) per layer, top-8. Experts are the param hog (per-layer MoE $\sim$ 11.3B, 96%); the dense skeleton (attention+emb) only $\sim$ 30B.

MoE property: each token activates only 8/256 experts $\rightarrow$ compute (fwd+bwd) both $\propto$ activated (37B); backward computes gradients only for activated experts (unselected have zero gradient this batch). **But memory isn't saved:** all 256 experts' params + AdamW must sit in memory on standby $\rightarrow$ EP splits experts across cards.

Back-and-forth ⑬ (does training compute save) — Once thought "MoE only saves at inference; training computes gradients for all 671B". **Correction:** backward only computes gradients for **activated experts** (unrouted experts have zero gradient). So training compute is also $\propto$ activated (37B), **not only inference**. But training **memory** isn't saved (all experts on standby).

5.3 Split experts + routing (the full-X-per-card scheme)

2048 cards = 256 nodes $\times$ 8. 256 experts across 256 nodes $\rightarrow$ 1 expert/layer per node. A node's 61-layer experts $\approx$ 2.69B params, $\sim$ 32G training, $\sim$ 4G/card across 8.

Back-and-forth ⑭ (TP-split experts? what comm?) — Proposed "each expert TP-split across 8 cards + ring all-reduce". **Correction:** mixed TP and EP. Experts are small (44M, fit one card); EP means "one expert whole in one place", not TP-split; and EP's comm isn't all-reduce.

Back-and-forth ⑮ (move experts vs move tokens) — Proposed "ring the experts, only the top-8 used". **Correction:** ① $2048 \times 8 = 16384$ selections over 256 experts, each expert selected **~64 times** on average → nearly all 256 experts used, "only top-8" $\approx$ still all. ② Moving one expert is 88MB, while its corresponding tokens average $64 \approx 0.9\text{MB}$. **Move whichever is smaller → move tokens (small), not experts (large), $\sim 100\times$ cheaper.** ③ Ring-moving experts traverses all used experts (serial, many steps); all-to-all moves tokens in one parallel step.

Each card has full X. Amortized: each expert averages **64 tokens** ($= s \times \text{top_k} / E = 2048 \times 8 / 256$).
 1. Each node's e_i **locally** picks from full X the ~ 64 tokens that chose e_i , computes their output (other tokens are 0 for e_i). Because each card has full X, **no need to move tokens in**.
 2. Each node produces a "partial contribution" ($b, s, 7168$, nonzero only for tokens that chose this expert).
 3. **Cross-node sum:** each token's D = sum of its 8 experts' (across 8 nodes) contributions; 256 partials summed = full D. Every card gets full D, $+X_1 = X_2$.

5.4 Amortized analysis (why ~ 64 tokens/expert)

$s=2048$, top-8 → total selections = 16384. Uniform → each expert expected = $16384 / 256 = 64$.
Formula: expected tokens/expert = (total tokens $\times$ top_k) / E = $s \cdot k / E$. Each node's e_i nonzero output $\approx 64 \times 7168 \times 2 \approx 0.9\text{MB}$. Imbalance (some experts over-selected) → uneven load → this is why MoE needs an aux loss for load balancing.

5.5 Communication volume

Each node outputs $\sim 0.9\text{MB}$ nonzero. Cross-node sum: each node sends/receives $\sim 0.9\text{MB}$, 256 nodes $\approx$ **230MB/layer**. Vs "move experts": 256 experts $\times$ 88MB $\approx 1.6\text{TB}$ — moving activations (230MB) is $\sim 100\times$ cheaper than moving experts (1.6TB). Sum uses a ring (stepwise accumulate, not gather-to-one-node).

Back-and-forth ⑯ (is full-X valid — the judge repeatedly forced "standard DP split", the judge's error) — You had set "independent/same data per card" early, but the judge repeatedly challenged with "standard MoE is DP-split, different tokens per card, needs two all-to-alls", forcing a different premise onto your self-consistent scheme. **After clarifying and apologizing:** under your setup (full X per card), tokens are all local, needing only one "move-results sum" (vs standard's move-in + move-out two all-to-alls). Cost is just redundant full X per card (small, $2048 \times 7168 \times 2 \approx 28\text{MB}$) + one router pass per card. **When X is small, this locality scheme wins (saves one global move).**

Conclusion: standard EP + DP → different tokens per card → two all-to-alls (move-in compute, move-out); **full-X-per-card → one move-results sum**. Which wins depends on X size: small X

(short context) $\rightarrow$ full $X + \text{locality}$; large X (long context) or large-scale DP $\rightarrow$ split. EP's comm is fundamentally all-to-all (token redistribution by target expert), unlike TP's all-reduce (sum).

6. FlashAttention: start from O's storage

6.1 Problem: $S \propto s^2$, can't store for long sequences

```
S = Q KT /  $\sqrt{d}$       # (n, m) scores,  $\propto s^2$ 
P = softmax(S)         # (n, m)
O = P V                # (n, d)
```

S is (n,m), memory $\propto s^2$. At $s=1M$, $S = 10^{12}$ elements — no memory holds it. **The core conflict of long context:** attention needs QK^T , but storing full S is infeasible.

6.2 Breakthrough: start from O, not S/P

Output **O is only (n,d)** — linear in s , not squared. O fits. So flip it: not "how to store S " but "how to compute O without storing full S ".

softmax (row i , minus max for stability): $m_i = \max_j S_{ij}$; $P_{ij} = \exp(S_{ij} - m_i) / \sum_k \exp(S_{ik} - m_i)$; $O_i = [\sum_j \exp(S_{ij} - m_i) V_j] / [\sum_k \exp(S_{ik} - m_i)]$.

Both numerator and denominator sum over key (j/k) $\rightarrow$ can accumulate blockwise.

Difficulty: ① the denominator needs all keys' exp sum (global), incomplete when blocked; ② the subtracted m_i is the global max, but when computing a block the later blocks aren't seen — **only the running max so far is known; a larger later value requires correcting the past.** This is what online softmax solves.

Back-and-forth ⑰ (analogy to KV cache?) — Felt "accumulating O blockwise resembles KV cache". **Correction:** opposite. KV cache = **store** (cache computed K/V for reuse, save recompute); FlashAttention = **don't store** (block-compute S , use to update O then discard, save storage). One stores-for-reuse, one computes-then-discards — opposite.

Back-and-forth ⑱ (approximate?) — Once thought "FlashAttention is approximate". **Correction: exact.** online softmax rescale guarantees final O = exact full-softmax value (float reordering may differ in last bit, mathematically equivalent). **The approximate one is linear attention** (Performer/Linformer: rewrite $\text{softmax}(QK^T)V$ as $\phi(Q)(\phi(K)^TV)$, compute $\phi(K)^TV$ ($d \times d$) to avoid $s \times s$, $O(s)$ compute but lossy). Contrast: **FlashAttention exact ($O(s^2)$ compute + $O(s)$ storage, saves storage not compute); linear attention approximate ($O(s)$ compute, lossy).**

6.3 Online softmax: running max/sum/O + rescale

```
init: m=-inf, l=0, O=0(vector)
for j=1..T:                # over key blocks (in Ring: each ring-arrived K/V segment)
    S_blk = Q_i · K_jT /  $\sqrt{d}$     # this block's scores (only this block, no full S)
```

```

m_new = max(m, rowmax(S_blk))
p = exp(S_blk - m_new)
scale = exp(m - m_new)
O = O*scale + p·V_j
l = l*scale + rowsum(p)
m = m_new
O_i = O / l
# this block's unnormalized weights (new max)
# correction factor for old
# rescale old O + add new block
# rescale old l + add new block

```

Why rescale works: running O and l were accumulated under old max m. A new block's larger m_new means past terms subtracted too little by (m_new-m), i.e. over-multiplied by $e^{(m_new-m)}$. Since O, l are **linear accumulations**, multiplying them by $e^{(m-m_new)}$ is equivalent to re-normalizing every past term by m_new — no need to revisit discarded old blocks. This is the essence: **one scalar rescale corrects the entire history**.

Back-and-forth 19 (bugs in the hand-written pseudocode) — Initial version had bugs: ① w_t forgot exp (used raw $S \times V$, should be $\exp(S-m_new) \times V$); ② O overwritten not accumulated (O_it = per t overwrites, should $O +=$ over keys); ③ wrong index (O written per key t, but O is per query i, key dim summed out); ④ old O not rescaled (only rescaled sum l); ⑤ tmp_sum wrong max. **Correction:** the big idea (blocking + running max/sum + rescale) is right, but the landing must be exact — 6.3 above is correct. What was rusty: details (exp, accumulate-not-overwrite, index, rescale old O too).

6.4 Multi-head unrolled

The pseudocode above is **single-head**. In practice multi-head (32), TP puts a subset per card (16). **Each head computes its own O independently** — heads have their own Q/K/V, no sharing, no comm. So wrap a head loop:

```

for h = 1..16:
    # this card's 16 heads
    for i (query block):
        m,l,O = -inf,0,0
        for j (key block):
            # Ring: ring-arrived segments
            ... online softmax(head h's Q_h,K_h,V_h) ...
        O_i^h = O / l

```

Multi-head points: ① each head computes independently (own Q/K/V, own online softmax), **no inter-head comm**; ② 16 heads = 16 independent FlashAttentions (parallel/batched, not necessarily serial); ③ each head's O^h concatenated into this card's output slice; ④ **merging heads is not inside attention** — it's all cards' head outputs concatenated through W_o (down-projection), which is where TP's all-reduce happens (§ 7, § 2).

7. Ring Attention: multi-card FlashAttention for 1M context

7.1 Why seq-split is mandatory

At 1M, one card can't even hold X: $X = 1M \times 7168 \times 2 \approx 13GB$ (before K/V, activations, params). So **split by seq (token)**: 8 cards each hold $1M/8 = 2^{17} \approx 130K$ tokens.

Conflict: in attention, one card's Q segment (130K queries) must inner-product with **all 1M tokens' K/V** (global). But each card only has its own K/V → **must fetch the other 7 segments' K/V**.

7.2 Ring-passing K/V = the inner j loop

```
# Ring Attention, one card's view (holds Q segment, initial K/V segment)
m, l, o = -inf, 0, 0
cur_kv = this card's K/V segment
for step = 1..8:                                # ring rotates 8 times
    S_blk = Q_local · cur_kvT / √d
    m_new = max(m, rowmax(S_blk))
    scale = exp(m - m_new)
    o = 0*scale + exp(S_blk-m_new)·cur_kv_v
    l = l*scale + rowsum(exp(S_blk-m_new))
    m = m_new
    cur_kv = ring_send_recv(cur_kv)             # pass K/V to next card, receive from prev
o_local = o / l
```

This is Ring Attention = multi-card FlashAttention: FlashAttention's "iterate key blocks" becomes "ring-arrived K/V segments"; online softmax's online accumulation carries over verbatim. Storage: each card stores only its X/O segment + current one K/V segment (linear), no full S, no all K/V.

Back-and-forth ② (ring send vs all-reduce) — Once said "ring-all-reduce to fetch K/V".

Correction: passing K/V is **ring send/recv (point-to-point, each card hands K/V to the next)**, **not all-reduce** (which is sum-reduction). No sum here — O_i is accumulated **locally per card** via online softmax; K/V just flows around the ring so each query sees it. The real all-reduce is elsewhere: **after Wo projection** (head-TP output merge, Wo row-split, partial O per card, all-reduce sum — § 2). So: **no all-reduce inside attention (each head independent)**; **ring passes K/V as data; all-reduce after Wo (head TP merge)**.

7.3 Two parallel dims stacked (seq-split + head TP)

1M training uses both: **seq-split (Ring)** — X split by token (X too big), ring K/V; **head TP** — heads split per card (§ 2). Full layer: **60-layer outer loop → per layer: each card holds its seq segment + assigned heads → ring K/V + each head online-softmax accumulate O → head outputs concat through Wo (all-reduce) → FFN split (all-reduce)**.

Backward (BP): more involved than forward — FlashAttention backward re-blocks S (not stored in forward) for gradients; Ring backward re-rotates K/V (or stores stats). But still matrix mul/add + communication symmetric to forward. The big structure is unchanged.

8. 100M: change the algorithm, framework unchanged

1M/10M: Ring+Flash holds (compute still $O(s^2)$ but splittable, tune via cards/blocks). **100M: $O(s^2)$ explodes** (standard attention's 100M KV cache alone needs hundreds of H100s/user) → **must change the algorithm** (linear/sub-quadratic sequence algorithm, e.g. Magic's LTM, $\sim 1000\times$ cheaper). **But the underlying framework is unchanged:** what changes is the attention algorithm; underneath it's still matrix mul + reduce + parallelism. In practice, **1M context + RAG is generally enough.**

9. KV cache economics at scale

One layer $K+V = s \times d \times 2(K,V) \times 2$ bytes. 1M, $d=4096$, uncompressed: 16GB/layer, 60 layers = **960GB**. GQA (K/V halved to 2048): 8GB/layer, 60 layers = **480GB**.

Back-and-forth ② (KV cache math) — First wrote " $1M \times 4096 \times 2$ " = 8GB. **Clarified:** that's **one part (K or V), one layer**. Full needs $\times 2$ (K,V) $\times L$ (layers). One layer $K+V = 16GB$, 60 layers 960GB (uncompressed). KV cache is stored per layer.

Back-and-forth ② (scale service memory — by concurrency, not DAU) — Once computed by "1M user $\times$ 100 session", got an astronomical number. **Clarified:** KV cache memory is used by the **concurrent requests currently decoding**, not DAU or total sessions (mostly idle). Estimate (1M DAU, 20 msgs/day, 10s/gen): avg concurrency ≈ 2300 , peak $\approx 7000-12000$. Avg 50K context (not full 1M), GQA: $\sim 6.5GB/request$, peak 10K concurrency $\approx$ **65TB $\approx$ 800 H100** just for cache. Everyone at full 1M $\rightarrow$ 1.3PB, impossible.

Conclusion: ① 1M context only for **paying/high-value users**; regular users short context + RAG; ② avg 50K $\rightarrow$ 800 cards for cache, + model replicas + compute + redundancy ($2-3\times$) $\approx$ 2400 cards, acceptable at 1M-DAU scale; ③ train/inference separated; ④ small models can co-locate model+KV cache on one card, large models split (TP) with KV cache distributed; ⑤ KV cache is the core memory bottleneck for large-scale long context $\rightarrow$ needs compression (MLA).

10. MLA: compress K/V jointly into a low-rank latent

Back-and-forth ③ (same as GQA?) — Once thought "MLA is the same idea as GQA". **Correction:** different. GQA = **reduce KV head count** (store fewer heads, still original K/V); MLA = **compress**

K/V into a low-rank latent (store compressed, decompress on use). GQA reduces heads, MLA compresses+reconstructs.

Back-and-forth ②④ (is MLA PCA?) — Once thought "MLA does PCA". **Correction:** no. PCA is data-driven (covariance eigenvectors, no learning); MLA is a **learned projection** — $W_{\text{down}}/W_{\text{up}}$ are trained parameters, end-to-end (like LoRA's learned low-rank projection), not PCA.

Back-and-forth ②⑤ (is MLA attention-matrix factorization?) — Once framed it as "factorizing the attention matrix". **Correction:** the abstract idea of low-rank compression is shared, but ① MLA compresses the **K/V vectors** (per token), not the attention matrix $S (QK^T)$; ② MLA **learns a projection**, not factorizes an existing matrix (PCA/SVD factorize). **Perspective: MLA is "compressing data information" (learned low-rank representation preserving K/V info), not a storage trick or matrix factorization.**

10.4 The math (compress + decompress)

Standard MHA: each token $x \rightarrow K = x \cdot W_k, V = x \cdot W_v$ (store two full-dim vectors).

MLA:

- **Compress (down-projection):** $c = x \cdot W_{\text{down}}$ — compress x into a low-dim latent c (e.g. dim 512, far smaller than full K/V). **Store only c (one, jointly encoding K,V).**
- **Decompress (up-projection):** on use $K = c \cdot W_{\text{up}_k}, V = c \cdot W_{\text{up}_v}$ — up-project K and V from the same c .

$W_{\text{down}}/W_{\text{up}_k}/W_{\text{up}_v}$ are all **learned parameters** (end-to-end, optimized with next-token), not PCA/SVD. Like Part2's LoRA (learning low-rank B/A), MLA learns low-rank projections.

Where it saves: standard stores $K+V$ two full-dim (e.g. 2×4096); MLA stores one latent c (e.g. 512). **KV cache drops from $2 \times d$ to latent dim**, $\sim 10\text{-}16\times$ compression. W_{up} can be absorbed into subsequent matrices (some implementations), saving more compute. **Cost:** an extra up-projection on use; but cache drops greatly + performance close to full MHA (less loss than GQA head reduction).

1M/60-layer memory: latent 512, one copy $\rightarrow 1\text{M} \times 512 \times 2 \approx 1\text{GB/layer}$, 60 layers $\approx 60\text{GB}$ (latent 400 $\rightarrow \sim 48\text{GB}$). Vs uncompressed 960G, GQA 480G, MLA $\sim$ a magnitude less. $\sim 6\text{-}8\text{G/card}$ across 8.

Back-and-forth ②⑥ (pass first or up-project first) — Under 1M seq-split + ring, MLA's attention compute is still matrix mul/add + reduce + Ring + Flash, plus one step "up-project latent $\rightarrow K/V$ ". **Order:** ring passes the **compressed latent c (small, saves bandwidth)**, then up-project locally to K/V , then attention. Pass-compressed-then-local-up-project beats up-project-then-pass-full- K/V (large bandwidth).

Perspective: GQA is a storage view (reduce heads); MLA is an information-compression view (learn a low-rank representation preserving K/V info, reconstruct on use). MLA compresses harder (close to full MHA, much smaller cache), key to memory-efficient long-context inference.

Summary

From "one card can't hold it": **TP** (head-split, up local / down all-reduce) → **grad ckpt** (s^2 activation, fine-grained recompute) → **DP** (independent data) → **EP** (full X + local compute + move results) → **FlashAttention** (start from O, online softmax, exact) → **Ring Attention** (1M, seq-split + ring K/V) → **100M** (change algorithm) → **KV cache economics** → **MLA** (joint low-rank latent).

The through-line: **matrix mul + reduce + parallel communication**. All scales and optimizations are splits and compressions on this framework under memory/bandwidth constraints. **And every back-and-forth in the derivation — from wrong assumption to correction — is the real path to understanding.**

— End of Part 3 (Detailed). 26 derivation back-and-forths. —